When Do Vulnerability Detectors Rely on Their Graphs? Evidence from Attention Biases, Adapters, and ReGVD

Anonymous Authors
Omitted for double-anonymous review

***Abstract*—Providing program graphs to a vulnerability detector does not establish that its predictions rely on authentic graph topology. We distinguish performance gains from separately trained graph controls and graph reliance measured by interventions on a fixed checkpoint. We examine three graph pathways on one corrected, component-controlled PrimeVul development cohort: CodeBERT with adaptive AST/data-flow attention biases, a graph-response adapter with and without matched-null subtraction, and the published ReGVD architecture adapted to the same cohort with its native token-co-occurrence graph. Across three matched seeds, authentic attention achieved 0.1342 ± 0.0169 average precision (AP), versus 0.1455 ± 0.0266 after graph-shuffled training; graph removal slightly increased AP for every authentic attention checkpoint. A null-centered adapter modestly improved separately trained sequence-only AP (0.17385 vs. 0.16587), although its direct correction added only 0.000298 AP at fixed checkpoints. Conversely, ReGVD's native graph improved AP over endpoint-permuted and self-only controls (0.15266 vs. 0.13724 and 0.14152) and showed measurable same-checkpoint sensitivity. Synthetic endpoint-alignment tests passed the attention criteria but exposed a failed adapter shuffled-training control in two seeds; the anomaly remains unexplained. This mixed evidence shows that topology reliance depends on architecture and intervention; predictive gains and synthetic success alone do not establish useful real-data topology reliance. Results concern the checkpoint-selection development split, not an independent final test.**

***Index Terms*—Vulnerability detection, graph reliance, program analysis, controlled experiments, negative results, graph neural networks.**

I. Introduction

A source-code vulnerability can depend on a definition reaching a use, an unsanitized value flowing into a sink, or a syntactic condition guarding an operation. Such dependencies motivate program-graph representations in neural vulnerability detection. Existing systems range from sequence-based CodeBERT and LineVul to data-flow-aware GraphCodeBERT and graph-neural models such as Devign and ReGVD [1]–[5]. Yet a graph input is neither a manipulation check nor evidence that the trained predictor exploits graph endpoint identities.

Consider a graph-augmented classifier that improves AP relative to another trained sequence model. Extra parameters, training dynamics, source length, degree profiles, or regularization can account for the improvement even if authentic edges are irrelevant. Conversely, an inference-time edge removal can change predictions without proving that the original edges carry reliable vulnerability information. We therefore separate *predictive utility*, *same-checkpoint reliance*, and *capacity to use structural alignment*.

Our study began with an AST/DFG-gated CodeBERT detector (ASTGuard) and developed into a comparative intervention study. An implementation audit subsequently discovered a consequential split-manifest shift in the historical campaign. We do not reuse its performance claims. The corrected campaign retains aligned source roles and makes positive, negative, and mixed results equally reportable. It includes a different graph family, ReGVD, precisely to avoid inferring that one attention mechanism characterizes all graph-aware detectors.

Contributions. (1) We operationalize three different questions about graph value with within-family training controls and fixed-checkpoint interventions. (2) On corrected real data, we show that adaptive AST/DFG attention does not reproducibly outperform shuffled training; null-centered adapters yield small between-run improvements with little direct AP contribution. (3) We report a positive, bounded counterexample from ReGVD, alongside a synthetic sensitivity test that succeeds for attention but is inconclusive for a centered-adapter shuffle control. (4) We disclose the split defect, role-repair protocol, non-independent evaluation, input-collision caveats, and intervention limitations rather than retrospectively treating old results as clean.

II. Research Questions and Interpretive Rules

RQ1: Predictive value. Does training with authentic graph endpoints outperform a matched model trained with perturbed endpoints (and a graph-free or self-only baseline)? Compare checkpoints trained independently under each family's held-constant recipe, reporting the paired seed differences in selection-split AP.

RQ2: Fixed-checkpoint reliance. How does replacing or removing the graph affect the *same* trained model's logits and rankings? These interventions measure response to an input channel under the tested perturbation; they do not identify a semantic causal effect of true dependencies.

RQ3: Sensitivity adequacy. Can the architecture learn a synthetic target defined only by graph-endpoint/token align-

ment, and do shuffled controls fail as expected? An architecture's ability on this isolated task cannot imply a real-world advantage.

The protocol was frozen before the corrected campaign: for attention, authentic-minus-shuffled AP must be positive in *all* three seeds with mean ≥ 0.02, and graph removal must produce mean absolute logit change ≥ 0.05. For ReGVD, the sign-consistent native-minus-perturbed mean threshold is likewise 0.02 AP. For the synthetic task, each seed requires authentic AP ≥ 0.90, authentic-minus-sequence and authentic-minus-shuffled AP ≥ 0.30, and authentic-minus-empty and authentic-minus-null AP $\geq 0.30/0.20$. These operational cutoffs are not significance tests, minimum important differences endorsed by the field, or equivalence margins. Three seeds admit descriptive direction checks only.

III. Models and Intervention Design

A. Adaptive AST/DFG attention

Let $H^{(\ell-1)}$ denote the hidden states before transformer layer ℓ. In attention head h, the pre-softmax score from token i to token j becomes

$$s_{ij}^{\ell h} = \frac{q_i^{\ell h}(k_j^{\ell h})^\top}{\sqrt{d_h}} + \sum_{r\in\{\mathrm{AST},\mathrm{DFG}\}} \beta_r^{\ell h}\, g_{ir}^{\ell h}\, M_{ij}^r, \qquad (1)$$

where M_{ij}^r indicates an aligned relation edge, β is a learned magnitude, and $g_{ir}^{\ell h} = \sigma(w_r^{\ell h\top} H_i^{(\ell-1)} + b_r^{\ell h})$ is a query-dependent gate. Ordinary attention is not hard-masked. Edges omit padding/special tokens and are truncated consistently with the token sequence. Disabling the structural term gives a sequence-only comparator. The real attention recipe uses structural layers 2, 5, 8, and 10 and 512-token maximum inputs; the synthetic sequence length is 26. Real attention trains for five complete epochs with weighted BCE and new-module learning-rate multiplier 50. The twelve-epoch synthetic attention experiment uses *unweighted* BCE and the same multiplier. Although its configuration declared an AST-only relation channel, the executed factory instantiated *AST and DFG gates*; synthetic DFG edges were empty, so the synthetic target signal was entirely in the AST channel. Matched graph-shuffled training uses length-matched donors (rather than node reindexing) while selected-checkpoint evaluation supplies authentic graphs, creating a contrast between training regimes. In both families this evaluation-on-authentic procedure also constrains interpretation: a shuffled-trained checkpoint can be selected for performing well on authentic tune graphs.

B. Uncentered and centered response adapters

A separate family computes a base sequence logit $b_\theta(x)$ and a graph response $r_\phi(H, G)$ over projected token features and relation-specific neighbors. Its conceptual uncentered logit is $b_\theta(x) + \alpha r_\phi(H, G)$, while the centered response is

$$z_{\mathrm{center}}(x, G) = b_\theta(x) + \alpha \left[r_\phi(H, G) - \frac{1}{K}\sum_{k=1}^{K} r_\phi(H, \widetilde{G}_k) \right], \qquad (2)$$

In the real-data centered recipe, $K = 4$ matched null graphs are generated with *four approximately equal position blocks*, not blocks of four tokens. Within each relation, eligible endpoints are permuted among nodes sharing *exact incoming degree, exact outgoing degree, and positional block*. This preserves constrained nuisance statistics, not program semantics. A 64-dimensional token projection forms AST- and DFG-specific neighbor summaries, with a zero-initialized scalar response head. The real sequence, uncentered, and centered conditions are separately trained for three epochs using weighted BCE and a new-module learning-rate multiplier of 10. The synthetic centered recipe differs: $K = 8$, *one* position block, AST-channel edges only, unweighted BCE, twelve epochs, and multiplier 50. Null identities at training/selection and at later FP32 intervention evaluation also use different seeded namespaces. A centered checkpoint's *sequence path*, *empty graph*, and *null substitution* remain different operations because null-response subtraction can persist when the authentic graph is removed.

C. ReGVD and its native graph

ReGVD [5] constructs unique-token, undirected window-five co-occurrence graphs, rather than ASTs or DFGs. The adapted implementation follows pinned upstream revision `b5e6657b003ada704b7c75e4bd237c1eee1d8138`: frozen GraphCodeBERT word embeddings, fixed 400-token inputs (including upstream padding behavior), normalized adjacency without self-loops, two residual ReGCN layers with 128 hidden units, and sigmoid soft attention with sum-times-max readout. It preserves the upstream GNN FP64 and classifier FP32 division, uses unweighted binary cross entropy, batch size 128, learning rate 5×10^{-4}, zero weight decay, and gradient clipping at one. Our *cohort and tune-AP checkpoint selection* differ from the upstream published benchmark and accuracy-selection protocol; these are not reproduction scores.

Native-graph training is compared with *endpoint-permuted* training that holds node degrees fixed while changing neighbors, and with *self-only* training that uses identity adjacency at both train and evaluation. Selected native checkpoints also receive permuted, null, empty, and self-only inference interventions. Node-order permutation alone is not a meaningful counterfactual because permutation-equivariant graph processing is invariant to consistent reindexing; our endpoint intervention instead changes adjacency relative to node-feature identity. All ReGVD settings use matched seeds and budgets.

D. What the two comparison types identify

For condition c and seed s, write $\hat{f}_{c,s}$ for the independently trained, tune-selected checkpoint. The between-training contrast is

$$\Delta_s^{\mathrm{train}}(a, b) = \mathrm{AP}(\hat{f}_{a,s}; D_t) - \mathrm{AP}(\hat{f}_{b,s}; D_t). \qquad (3)$$

At a *fixed* checkpoint $\hat{f}_{a,s}$, an intervention instead gives

$$\Delta_s^{\mathrm{within}}(G, G') = \mathrm{AP}(\hat{f}_{a,s}(x, G); D_t) - \mathrm{AP}(\hat{f}_{a,s}(x, G'); D_t). \qquad (4)$$

We additionally inspect $n^{-1}\sum_i |z_i(x_i, G) - z_i(x_i, G')|$. AP is non-interpolated average precision with tied scores grouped. Differences in separately trained models are *not* decompositions of a checkpoint's prediction; an adapter can affect learning without contributing much of the selected checkpoint's final logit.

IV. Data, Integrity, and Experimental Protocol

A. Corrected cohort and selection role

The development cohort derives from PrimeVul [6], a benchmark designed with chronological and deduplication safeguards and severe class imbalance. The corrected manifest has 141,560 training functions, of which 4,302 are positive, and 10,566 *tune* functions, of which 330 are positive (3.12% tune prevalence). Its identifier is `793ac1b2bcacb06aa19aef62ac575482dbb76a5307d026997ac239ce5157f5c2`.

During the historical campaign, deleting five outliers shortened only an ordered-ID column while leaving row-aligned roles, component IDs, and original splits unchanged. This shifted 10,619 role assignments, including 2,612 original-test samples mapped to train, and introduced cross-component contamination. All historical real-data performance aggregates are excluded. The new cohort is built from the aligned original, filtering every row-indexed column, rehashing the manifest, and verifying original split roles, components, source hashes, sample IDs, and feature inputs. New experiments consume corrected train/tune only. We *do not* claim that the historically referenced official test remains pristine or offer any independent final-test estimate.

A rebuilt-population scan found no reported raw-source overlap but detected nine token-sequence fingerprints and eight local-model-input fingerprints shared between corrected train and tune, consistent with collisions after truncation/projection; strict input-level disjointness is therefore *not* claimed. Lexical fingerprints were not fully stored, preventing a complete lexical-clone audit. Tune extraction statuses include 6,933 AST/DFG-both-OK, 2,231 AST-OK/DFG-unsupported, 1,284 both-parse-failed, 80 parse/lexical-failed, and 38 AST-OK/lexical-failed. Examples remain in evaluation under the declared empty/status policy, rather than disappearing after extraction failures.

B. Campaign matrix and measurement

The experiment matrix comprises 42 completed training jobs: nine five-epoch real-data attention jobs, nine three-epoch real-data adapter jobs, nine ReGVD jobs, and fifteen synthetic jobs. The ReGVD ceiling is 100 epochs, with at least 20, patience ten tune-AP improvements, and minimum improvement 10^{-4}. Three matched seeds, $\{42,123,456\}$, are retained for every cell. Within each family, cohorts, preprocessing, selection rule, loss, model revision, optimizer recipe, and training budgets are controlled. Different attention and adapter family schedules create different sequence baselines: we do not rank the two family means against one another.

TABLE I
Executed training recipes, not interchangeable between real and synthetic or between model families. The position parameter counts blocks, not tokens per block.

Family	Loss / duration	Structural and precision details
Real attention	Weighted BCE; 5 ep.; multiplier 50	AST+DFG, layers 2/5/8/10; 512 tokens; BF16 train
Real adapter	Weighted BCE; 3 ep.; multiplier 10	AST+DFG; $K = 4$, 4 position blocks; BF16 train
Synthetic attention	Unweighted BCE; 12 ep.; multiplier 50	AST+DFG gates executed, DFG empty; 26 tokens
Synthetic adapter	Unweighted BCE; 12 ep.; multiplier 50	AST only; $K = 8$, 1 position block
ReGVD	Unweighted BCE; 20–100 ep. bound	Window-5 token graph; 400 tokens; GNN FP64/classifier FP32

The local families use pinned CodeBERT revision `3b0952feddeffad0063f274080e3c23d75e7eb39`, BF16 training/selection and a common backbone/feature recipe *within each family*. Real-data local models use prevalence-weighted BCE; synthetic experiments use unweighted BCE. The preserved local backbone learning rate is 2×10^{-5}, with family-specific new-module multipliers specified above. The real local-model recipes use microbatches of four and gradient accumulation of eight (effective batch 32); the audit confirms total optimizer steps of 22,120 for five-epoch real attention and 13,272 for three-epoch real adapters. Exact warmup/resolved optimizer settings beyond the supplied frozen protocol must be checked against per-run configurations prior to an independent reproduction; they are not reconstructed from memory in this manuscript. The ReGVD optimizer and precision are different (Section III).

We report non-interpolated tune-selection AP (float32 sigmoid probabilities), arithmetic means and sample SD, and all matched-seed differences. Fixed-checkpoint intervention AP can use raw logits in FP32, while centered interventions additionally use an evaluation-specific null namespace. These differences can induce small AP drift relative to the selected score. They do *not* invalidate a within-evaluation full-minus-base contrast. The independent October 9 audit recomputed selected AP from raw predictions for 42 jobs and AP, ROC-AUC, and mean absolute logit change for 156 intervention-condition results, finding no discrepancies above 10^{-12} for the checked AP comparisons. It also checked selected-checkpoint hashes against retained manifests/reports and selected-epoch prediction consistency. This is numerical and hash consistency, *not* retraining or a fresh weights-to-predictions reproduction. The frozen response thresholds $0.01, 0.02, 0.05, 0.10$ define sensitivity diagnostics and are not equivalence cutoffs.

V. Results

A. RQ1: training with authentic topology

Attention: no consistent authentic advantage. As Tables II–III show, authentic minus shuffled is negative in seeds 42 and 123 and positive in seed 456; the mean is -0.01129 AP. Authentic minus sequence has a mean $+0.01475$ AP but is also negative in two seeds. Neither contrast passes the frozen sign-and-effect threshold. The three-seed descriptive Student-t

TABLE II
CORRECTED REAL-DATA TUNE AP (MEAN $\pm$ SAMPLE SD; $n = 3$ SEEDS). ROWS COMPARE ONLY WITHIN THE INDICATED TRAINING FAMILY. ALL VALUES ARE CHECKPOINT-SELECTION OUTCOMES, NOT INDEPENDENT TEST ESTIMATES.

Family	Trained condition	Tune AP $\pm$ SD
Attention, 5 ep.	Sequence only	0.11943 ± 0.01465
	Authentic AST/DFG	0.13418 ± 0.01693
	Length-matched shuffled	0.14547 ± 0.02661
Adapter, 3 ep.	Sequence only	0.16587 ± 0.00580
	Uncentered	0.16965 ± 0.00234
	Null-centered	0.17385 ± 0.00594
ReGVD	Native co-occurrence	0.15266 ± 0.00071
	Endpoint-permuted	0.13724 ± 0.00270
	Self-only	0.14152 ± 0.00196

TABLE III
MATCHED-SEED TRAIN-CONTRAST AP DIFFERENCES: $s = 42, 123, 456$, FOLLOWED BY ARITHMETIC MEAN. A POSITIVE MEAN ALONE NEVER SUBSTITUTES FOR THE PRESPECIFIED ALL-POSITIVE AND ≥ 0.02 DECISION RULE.

Contrast	42	123	456	Mean
Attn authentic $-$ shuffled	$-.02161$	$-.04873$	$+.03648$	$-.01129$
Attn authentic $-$ sequence	$-.00199$	$-.00276$	$+.04901$	$+.01475$
Centered $-$ sequence	$+.00661$	$+.00864$	$+.00870$	$+.00798$
Native $-$ permuted	$+.01440$	$+.01783$	$+.01405$	$+.01543$
Native $-$ self-only	$+.01398$	$+.01006$	$+.00940$	$+.01115$

interval for authentic-minus-shuffled is $[-0.11943, +0.09686]$: it is too wide to support an equivalence conclusion. The corrected longer-budget runs therefore fail to establish robust authentic-topology benefit under this particular recipe, not zero possible effect.

Adapters: a modest between-training gain. The centered model beats the family-matched sequence-only model in all three seeds by 0.00661, 0.00864, and 0.00870 AP, a mean of 0.00798 (about 0.80 AP percentage points). It also exceeds the separately trained uncentered mean. The result misses the frozen 0.02 AP magnitude criterion. Its sign consistency is real for these three runs, but three seeds and a tune-only evaluation do not license a population-level superiority claim.

ReGVD: a bounded positive counterexample. Native co-occurrence graphs beat endpoint-permuted graphs in all seeds (mean $+0.01543$ AP) and self-only graphs in all seeds (mean $+0.01115$ AP). Both effects miss the predeclared 0.02 criterion, although descriptive paired intervals are $[+0.01023, +0.02062]$ for native-minus-permuted and $[+0.00500, +0.01730]$ for native-minus-self. These selected-cohort/three-seed intervals do not quantify unseen-dataset uncertainty. The directional differences contrast with attention's mixed signs. ReGVD's graph is not an AST/DFG graph, and its optimization is different; this is evidence of architecture-and-graph-family heterogeneity, *not* a controlled cross-family ranking of models.

B. RQ2: same-checkpoint intervention evidence

For the authentic attention checkpoints, removing all structural edges slightly *increased* AP at each seed in the provided intervention handoff. The mean absolute logit changes under empty-graph evaluation at seeds 42, 123, and 456 are 0.00672, 0.01948, and 0.03038, respectively; all are below the prespecified 0.05 threshold. Hence authentic attention is neither consistently more predictive than matched shuffling nor strongly sensitive under the designated complete-removal operation. The prespecified cutoff sequence $0.01/0.02/0.05/0.10$ classifies these three mean absolute responses as 2/1/0/0 above cutoff, respectively (strict exceedance); changing an arbitrary cutoff changes the interpretation. They must not be conflated with a weaker matched-null intervention.

At selected centered adapter checkpoints, the three *full-minus-sequence-path* AP contributions in one paired FP32 intervention evaluation are $+0.00033628$, $+0.00031805$, and $+0.00023949$ (seeds 42/123/456; mean $+0.00029794$). The separately trained centered-minus-sequence mean is $+0.00798090$; these are different estimands. Full intervention AP also differs from each checkpoint's selected tune AP by $+0.00031828$, $+0.00020280$, and $+0.00011570$, respectively. Intervention precision, score representation, and null namespace differ from selection, and this drift is on a similar scale to the small direct contribution; it cannot be attributed entirely to ties or treated as evidence of graph utility. A training-mediated mechanism is *compatible* with the observations but not causally established. An empty graph, null-substituted graph, and direct sequence-path readout cannot be interchanged under Eq. (2). For real-data matched-null controls, approximately 18.6% of AST and 9.1% of DFG edges changed; they are not complete graph destruction.

ReGVD native checkpoints give stronger evidence of input-graph response. Evaluating the same native-trained model after endpoint permutation decreases AP in seed 42 from 0.15337 to 0.12828, in seed 123 from 0.15196 to 0.10108, and in seed 456 from 0.15266 to 0.13032. These native-to-permuted AP drops are 0.02509, 0.05088, and 0.02234 (seeds 42/123/456). Mean absolute logit changes are approximately 0.63180, 0.68619, and 0.55055, and about 57.9% of native edges change under endpoint permutation. This fixed-checkpoint sensitivity complements the independent training controls and contradicts the claim that every graph path is inert. It does not show that the changed edges have a security-semantic causal interpretation.

Audit boundary. The October 9 independent audit recomputed all 42 selected APs and 156 intervention-condition results from per-example predictions, verified population/label alignment, checked selected checkpoint hashes, and reported 114 passing repository tests. It separately verified synthetic split construction and confirmed that the corrected real-data evaluation streams contain 10,566 examples with 330 positives. The audit did *not* re-run full inference from the hashed weights, retrain the models, independently recreate all global near-clone components, or provide an anonymous complete artifact. The public repository's general README may describe an older campaign and is not, by itself, proof of a frozen corrected release.

TABLE IV
PAIRED INFERENCE-TIME RESULTS ON CORRECTED REAL TUNE (SEEDS 42/123/456). TRAINING CONTRASTS IN TABLE III ARE NOT INTERCHANGEABLE WITH THESE WITHIN-CHECKPOINT MEASUREMENTS.

Metric	42	123	456
Attention $\lvert\Delta z\rvert$, empty graph	.00672	.01948	.03038
Centered ΔAP, full–base	+.000336	+.000318	+.000239
ReGVD AP, native checkpoint	.153374	.151960	.152656
ReGVD AP, endpoints permuted	.128285	.101078	.130319
ReGVD $\lvert\Delta z\rvert$, permuted	.63180	.68619	.55055

TABLE V
SYNTHETIC HELD-OUT AUDIT AP, SEEDS 42/123/456, INDEPENDENTLY RECOMPUTED FROM SAVED PER-EXAMPLE PREDICTIONS IN THE OCTOBER 9 AUDIT. AUTHENTIC-MINUS-SHUFFLED CENTERED CRITERION FAILS AT SEEDS 123 AND 456.

Audit condition	42	123	456
Sequence-only	0.5000	0.5000	0.5000
Attention authentic	1.0000	1.0000	1.0000
Attention shuffled train	0.5270	0.5480	0.5690
Attention empty at authentic	0.5000	0.5000	0.5000
Attention null at authentic	0.5286	0.5343	0.5556
Centered authentic	1.0000	1.0000	1.0000
Centered shuffled train	0.5430	0.9994	1.0000

C. RQ3: synthetic endpoint-alignment sensitivity

The synthetic task creates pairs with identical token sequences and opposite labels. Both classes share unlabeled graph topology and degree profiles, but the label depends on whether graph edges connect equal or unequal token types. Graphs occupy the AST input channel but are *not parsed ASTs*; these scores do not represent vulnerability detection. Train, tune, and synthetic audit contain 4,096/1,024/1,024 examples, respectively, with disjoint token sequences. All fifteen runs use twelve full epochs (1,536 steps); checkpoints are selected on synthetic tune only, and synthetic audit is evaluated afterward.

The attention model meets *all five* predeclared success checks in each seed: authentic audit AP is one, the sequence model is at chance, training-time shuffling collapses performance near chance, and empty/null interventions degrade the same authentic checkpoint. The architecture can therefore learn a target that genuinely requires the specified endpoint/token alignment under the isolated conditions.

The centered adapter likewise achieves authentic audit AP of one, but its shuffled-training checkpoints score 0.9994 and 1.0 in seeds 123 and 456. Thus authentic-minus-shuffled falls below 0.30 in two seeds and the complete validation criterion *fails*, even though the authentic checkpoint's own empty/null interventions pass the frozen checks. The repaired centered-shuffled execution path nests the matched-null wrapper around the graph-donor wrapper around the original feature dataset; the earlier wrapper-overwrite defect therefore does not explain this corrected failure. The independent audit confirmed disjoint synthetic sequences, class-balanced pairs and similar simple donor-transplant alignment rates between labels (roughly 21.5–21.9%), but these checks cannot exclude every higher-order cue. In particular, shuffled-training checkpoints were selected using *authentic* tune graphs, which can favor an authentic-sensitive checkpoint without establishing how it arose. Full real/null pathway inspection and training trajectories remain undiagnosed. No favorable reruns are substituted, and no post-hoc analysis is represented as prespecified. This failure blocks a claim that the centered adapter passes the complete sensitivity validation.

VI. DISCUSSION

A. What constitutes a meaningful graph result?

Our findings resist a binary story. Weak attention response is specific to the frozen backbone, relation extraction, input length, budget, and interventions. The adapter's between-training improvement does not, by itself, demonstrate use of authentic endpoints at inference. ReGVD offers a positive comparison in which native graph training and fixed-checkpoint endpoint perturbation point in the same direction. Graph reliance is therefore both *architecture-dependent* and *intervention-dependent*. Relying on a graph may mean using degree, edge density, endpoint-token alignment, or other correlated statistics; only careful counterfactual controls can progressively narrow the explanation.

B. Lessons for experimental design

First, use graph identity-preserving controls that change the intended relationship, rather than mere node-order permutations. Second, compare independently trained shuffled controls *and* same-checkpoint interventions; neither substitutes for the other. Third, validate intervention strength: a matched null with approximately 9% DFG-edge changes is not a graph deletion. Fourth, establish a synthetic alignment-sensitive task with a separate audit partition but treat unexpected shuffled-model success as a failed control. Fifth, audit data roles and caches before interpreting any exact metric: correctly recomputed AP from a mislabeled split is still invalid evidence for the intended protocol.

C. Interpreting RENE-style mixed results

The informative negative conclusion is not that program graphs fail categorically. It is that *a purportedly structural model needs evidence of reliance beyond predictive score and architecture name*. ReGVD prevents unwarranted universality, and the imperfect synthetic adapter control prevents overstating mechanism certification. The actionable outcome is an intervention-oriented test suite and precise limits on what the three families demonstrate.

VII. THREATS TO VALIDITY

Selection and power. All real metrics use the 10,566-example tune split for checkpoint selection and reporting; performance is optimistically selected and cannot be interpreted as final-test generalization. Three seeds do not yield narrow uncertainty estimates; threshold nonattainment is not an equivalence test. Five attention epochs do not establish convergence: attention seeds 123/456 select epoch five and every real adapter run selects epoch three, at the budget

boundary. The current package lacks full per-epoch prediction/loss histories for independent learning-curve replotting. An additional dataset and true deployment evaluation remain untested.

Historical contamination. The original experiment manifest was shifted, and historical official-test samples reached training. We quarantine all earlier real-data claims and explicitly decline to reuse the historical official test as untouched evidence. The corrected manifest ensures intended role mappings for the new campaign but does not undo historical exposure. The nine sequence and eight projected-input train/tune collisions and incomplete lexical-clone hash fields limit claims of strict input disjointness.

Representations and interventions. AST/DFG extraction frequently returns absent/unsupported relations and follows a declared empty policy. Truncation can erase useful dependencies. Length-matched donor shuffling and block-matched nulls can preserve nuisance statistics or residual authentic alignments; ReGVD's window-five graph is not an AST/DFG substitute. The synthetic graphs are constructed relations, not vulnerable programs. The centered-shuffled anomaly remains unexplained.

Measurement and reproducibility. Within-family contrasts are matched, but attention, adapter, and ReGVD differ in architectures, recipes, and losses. Selection uses float32 sigmoid-based AP; FP32 intervention scores may use raw logits, and centered interventions use a different null namespace. Historical prefix-only attention-gate summaries have disproportionate positives and cannot support full-cohort statements. All 42 selected and 156 intervention results were independently recomputed from saved predictions, and selected checkpoint checksums were cross-checked, but full inference from weights and independent retraining remain untested. The October 9 audit compared 157 Python files across local and server snapshots; two differ in archival manifest fallback/default audit paths. Six early synthetic sequence/centered-authentic jobs lack the newer start-of-run provenance fields, while ReGVD lacks the same per-run source/environment trace as local models. A complete anonymous reviewer-run artifact and independent confirmation on unexposed real functions remain unavailable.

VIII. Related Work

Code-language models and structural guidance. CodeBERT represents code with a pretrained transformer [1]; GraphCodeBERT incorporates data-flow structure in pretraining and attention [3]. Their existence motivates structural inductive bias but does not answer whether *a particular fine-tuned detector* relies on authentic edges rather than graph-correlated nuisance features. Our focus is controlled evidence about trained pathways, not a new superior code encoder.

Feature reliance and code perturbations. Das et al.'s ICSE 2025 study, *Are We Learning the Right Features?*, defines feature-preserving and feature-eliminating code perturbations to test vulnerability and spurious-feature reliance across five vulnerability detectors [7]. That work asks whether classifiers respond appropriately when *program features* are retained or removed. Our controls instead compare the relative *alignment of supplied graph endpoints to fixed code tokens* under independent training and within-checkpoint intervention; they do not preserve code semantics and cannot replace feature-level causal tests.

Counterfactual graph explanation and faithfulness. CF-Explainer searches for local, minimal graph changes that flip a vulnerability-model decision across four GNN architectures [8]. GraphFramEx systematizes graph explanation sufficiency/necessity and fidelity, principally for node classification [9]. We do not introduce counterfactual graph explanation or degree-constrained permutations as a new primitive. Our contribution is a population-level conjunction of matched *training* controls, fixed-checkpoint perturbations, nuisance-matched nulls and synthetic sensitivity checks; unlike CFExplainer, the question is not which smallest local modification explains one decision.

Vulnerability detectors and benchmarking. Devign uses program graphs [4], ReGVD constructs token graphs and residual GNNs [5], and LineVul demonstrates the value of transformer representations for vulnerability prediction and localization [2]. PrimeVul documents evaluation hazards from data quality, de-duplication, chronology, and rare positives [6]; DiverseVul identifies broader dataset-generalization and false-positive challenges [10]. Rather than recomputing their original leaderboards, we compare controlled variants on a corrected common *development* cohort. No direct Devign, LineVul, GraphCodeBERT detector, or DiverseVul transfer score is produced by this frozen 42-job matrix, and none is claimed.

IX. Conclusion

Graph availability, authentic-graph training, and inference-time graph reliance are three different properties. On corrected PrimeVul development data, CodeBERT's AST/DFG attention does not consistently profit from authentic edges relative to shuffled training; a null-centered adapter has a small between-training gain but little direct AP contribution. ReGVD nevertheless benefits modestly from native token-co-occurrence adjacency and responds substantially to endpoint permutation. Synthetic attention succeeds on a deliberately alignment-dependent task; the centered adapter's shuffled control fails in two seeds. These results favor a constrained, falsifiable approach to graph dependence, not a claim of graph uselessness or a new state-of-the-art detector. Independent unexposed real-data confirmation, an explanation or explicit bound for the failed synthetic control, and an anonymized independently runnable artifact are the next gates before a submission-ready claim.

Reproducibility and Artifact Status

The supplementary authoring package contains the frozen protocol, corrected 42-job selected-results summary, individual seed values, and the independent audit's written scope. The independent October 9 audit reports raw-prediction metric recalculation for 42 training jobs and 156 interventions, with

TABLE VI
CODEBERT REAL-DATA FAMILIES: SEED-LEVEL SELECTED TUNE AP, DIRECTLY EXTRACTED FROM THE CAMPAIGN JSON.

Condition	42	123	456
adpt:sequence	0.1655708	0.1718109	0.1602240
adpt:uncentered	0.1695233	0.1720497	0.1673772
adpt:centered	0.1721795	0.1804471	0.1689218
attn:sequence	0.1325456	0.1221186	0.1036201
attn:authentic	0.1305605	0.1193611	0.1526274
attn:shuffled	0.1521688	0.1680934	0.1161460

TABLE VII
REGVD: SEED-LEVEL SELECTED TUNE AP, DIRECTLY EXTRACTED FROM THE CAMPAIGN JSON.

Condition	42	123	456
regvdnative	0.1533743	0.1519601	0.1526563
regvdpermuted	0.1389773	0.1341265	0.1386049
regvdself	0.1393943	0.1418970	0.1432580

checkpoint/hash consistency, but the supplied ZIP *does not contain* all raw predictions, checkpoints, resolved configurations, or a tested anonymous runnable snapshot. We therefore make *no claim* of a complete publicly released reproducibility artifact, a successful independent weight-to-inference reproduction, or a verified anonymous review link. A named public development repository cannot substitute for an anonymous reviewed artifact. For double-anonymous review, a separate complete anonymized archive with source snapshot, manifest, run configs, environments, raw predictions, checksums, and audit commands must be tested and deposited; that step is outstanding.

APPENDIX A
ALL SELECTED TUNE AP VALUES

The following three tables enumerate all 42 completed *training jobs*, with one selected tune AP per logical job. These data come directly from the supplied summary JSON, not the separate intervention/audit handoff. Synthetic tune scores must *not* be substituted for synthetic audit scores in Table V.

APPENDIX B
FROZEN CONTROLS, CAVEATS, AND TRACEABILITY

The manifest identifier for the corrected population is 793ac1b2bcacb06aa19aef62ac575482dbb76a5307d026997ac239ce5157f5c2. The protocol used seeds {42,123,456} and no post-outcome hyperparameter search. The ReGVD adaptation follows upstream revision b5e6657b003ada704b7c75e4bd237c1eee1d8138; the nine ReGVD runs stopped between epochs 20 and 26, with selected epochs 7–16, none reaching the 100-epoch ceiling. Attention seeds 123/456 selected the fifth epoch; all real-adapter jobs selected epoch 3. These endpoint selections do not prove convergence. Nonidentical cohort and AP selection disallow claims of upstream score replication. The synthetic audit is held out *within the synthetic experiment* and is neither the official PrimeVul test nor confirmation

TABLE VIII
SYNTHETIC: SEED-LEVEL SELECTED TUNE AP, DIRECTLY EXTRACTED FROM THE CAMPAIGN JSON.

Condition	42	123	456
syn:sequence	0.5000000	0.5000000	0.5000000
syn:attn:authentic	1.0000000	1.0000000	1.0000000
syn:attn:shuffled	0.5267032	0.5471906	0.5702259
syn:centeredauthentic	1.0000000	1.0000000	1.0000000
syn:centeredshuffled	0.5416565	0.9998619	1.0000000

of semantic program dependence. Raw-prediction metric and selected-checkpoint hash checks were performed in the October 9 audit; full independent weight-to-prediction and training reproduction, an anonymous archive, and a new unexposed real-data evaluation remain open.

REFERENCES

[1] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "Codebert: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1536–1547. [Online]. Available: https://aclanthology.org/2020.findings-emnlp.139/

[2] M. Fu and C. Tantithamthavorn, "Linevul: A transformer-based line-level vulnerability prediction," in *Proceedings of the 19th International Conference on Mining Software Repositories*, 2022, pp. 608–620. [Online]. Available: https://doi.org/10.1145/3524842.3528452

[3] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, M. Tufano, S. K. Deng, C. Clement, D. Drain, N. Sundaresan, J. Yin, D. Jiang, and M. Zhou, "Graphcodebert: Pre-training code representations with data flow," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://arxiv.org/abs/2009.08366

[4] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," 2019. [Online]. Available: https://arxiv.org/abs/1909.03496

[5] V.-A. Nguyen, D. Q. Nguyen, V. Nguyen, T. Le, Q. H. Tran, and D. Phung, "Regvd: Revisiting graph neural networks for vulnerability detection," 2021. [Online]. Available: https://arxiv.org/abs/2110.07317

[6] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, and Y. Chen, "Vulnerability detection with code language models: How far are we?" 2024. [Online]. Available: https://arxiv.org/abs/2403.18624

[7] S. Das, S. T. Fabiha, S. Shafiq, and N. Medvidovic, "Are we learning the right features? a framework for evaluating dl-based software vulnerability detection solutions," in *Proceedings of the 47th International Conference on Software Engineering (ICSE)*, 2025. [Online]. Available: https://arxiv.org/abs/2501.13291

[8] Z. Chu, Y. Wan, Q. Li, Y. Wu, H. Zhang, Y. Sui, G. Xu, and H. Jin, "Graph neural networks for vulnerability detection: A counterfactual explanation," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024. [Online]. Available: https://arxiv.org/abs/2404.15687

[9] K. Amara, Z. Ying, Z. Zhang, Z. Han, Y. Zhao, Y. Shan, U. Brandes, S. Schemm, and C. Zhang, "Graphframex: Towards systematic evaluation of explainability methods for graph neural networks," in *Proceedings of the First Learning on Graphs Conference*, ser. Proceedings of Machine Learning Research, vol. 198, 2022, pp. 44:1–44:23. [Online]. Available: https://proceedings.mlr.press/v198/amara22a.html

[10] Y. Chen, Z. Ding, L. Alowain, X. Chen, and D. Wagner, "Diversevul: A new vulnerable source code dataset for deep learning based vulnerability detection," in *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses*, 2023, pp. 654–668. [Online]. Available: https://doi.org/10.1145/3607199.3607242